

Administrator Guide

Installing it, setting it up, wiring the notifications, and keeping the data safe on the day something goes wrong.

For whoever owns the installation

Covers deployment, configuration, notifications, access levels, backups and recovery

What is in this guide

1 How it is put together

2 Installing it

3 First run

4 Access levels and accounts

5 Branding and the site clock

6 Visitor types and the kiosk home screen

7 The details form, per type

8 Documents and questions

9 Induction slideshows

10 Badges and printers

11 Devices

12 Notifications

13 Designing the chat card

14 Doors and access control

15 Projects, companies and certificates

16 Backups and restoring

17 Disk space and retention

18 Security posture

19 Tests

20 Troubleshooting

How it is put together

One Node process and one SQLite file. There is no database server to run, no message queue, no build step, and nothing to rebuild when Node updates.

```
server/  
  index.js      express app, static hosting, retention and auto-sign-out jobs  
  db.js         SQLite schema and query helpers (node:sqlite, no native mod  
  settings.js   typed defaults, deep-merged over stored overrides  
  auth.js       cookie sessions, bcrypt passwords  
  roles.js      who can see what – the single place that decides  
  notify.js     email and chat webhooks  
  notify-card.js the chat card model and its renderers  
  expected.js   people booked in before they arrive  
  backup.js     archives, verification, the restore drill  
  storage.js    disk usage and the photo pressure valve  
  badges.js     badge numbering, shared by sign-in and reprinting  
  localtime.js  the site's day, for everything that counts one  
  routes/  
    kiosk.js    public kiosk API  
    admin.js    authenticated dashboard API  
    board.js    the on-site board  
  public/  
    kiosk/      the reception app  
    admin/      the dashboard  
    shared/     the common stylesheet
```

Where the data lives

```
data/smartlobby.db    the database  
data/uploads/public/  logo, induction slides      (served without login)  
data/uploads/private/ photos, signatures, parcels (login required)  
data/backups/         nightly archives
```

Back up that folder and you have backed up the system. Copy it to another machine and the system moves with it.

ONE INSTANCE ONLY

The database is a single file. Do not scale to multiple replicas — two processes writing one SQLite file is how a database gets corrupted.

Installing it

```
npm install
npm start
```

Kiosk at `/kiosk/`, dashboard at `/admin/`, and a device check page at `/check/` that reports what a given tablet's browser actually allows. Use `PORT=3010 npm start` for a different port.

Shape one: a tablet on your own network

The tablet runs the kiosk in its browser; the server runs on a small always-on machine on the same network — a Mac mini, a NUC, an old laptop, a Raspberry Pi. An iPad cannot host the server itself.

- 1 On the host machine, `npm install && npm start`.
- 2 Find its LAN address, for example `192.168.1.40`.
- 3 Register the tablet under **Devices** and press **Copy link** — it looks like `http://192.168.1.40:3000/kiosk/north-gate-ipad`.
- 4 Open that on the tablet, then Share → **Add to Home Screen**. Launched from the home screen it runs full screen, and because the device is named in the address itself the saved icon always comes back showing that device's cards.
- 5 Turn on **Guided Access** so visitors cannot leave the app.

THE CAMERA OVER PLAIN HTTP

Browsers only allow a live camera preview over `https://` or on `localhost`. Over a plain LAN address the kiosk automatically falls back to an **Open camera** button that launches the tablet's own camera app — the photo is captured, cropped and attached identically. Nobody hits a dead end. To get the inline preview, put a reverse proxy with a certificate in front (Caddy does it in two lines).

THIRD-PARTY KIOSK BROWSERS

A kiosk browser app must both hold the camera permission itself *and* forward the page's request. Ones that skip the second part deny it silently, with no prompt at all. If no prompt ever appears, that is the cause. Safari with Guided Access is the reliable alternative, and `/check/` on the tablet confirms it either way.

Shape two: Railway

- 1 Push the repository and create a Railway project from it.

- 2 Add a **Volume** and mount it at `/data` .
- 3 Set `NODE_ENV=production` and `PUBLIC_URL=https://your-app.up.railway.app` . `DATA_DIR` is optional — a Railway volume is detected automatically.
- 4 Deploy. Railway serves HTTPS, so the kiosk camera works out of the box.

THE VOLUME IS NOT OPTIONAL

Without it, the database and every uploaded photo and slide is erased on each deploy. The app refuses to be quiet about this: a banner appears in the dashboard, the login screen says so, `/api/health` reports `"storage": "ephemeral"` , and the deploy log prints a warning.

The included Dockerfile installs LibreOffice, poppler and metric-compatible fonts, so uploaded PowerPoint decks render as they look in PowerPoint. It makes the image about a gigabyte and the first build slow. Delete it if you would rather have fast builds and are happy uploading PDF exports of your decks instead.

First run

The first time you open `/admin/` it asks you to create the owner account. That is the whole installation — there is nothing else to provision.

A new install is not an empty screen: it seeds one worked example per visitor type so you can see what a visit, a contractor sign-in, an interview and a driver record actually look like. Clear them from the dashboard when you are ready. They are flagged in the database rather than matched by name, so clearing them can never touch a real visitor who happens to share a name.

The order worth setting things up in

- 1 **Settings** → **Branding** — the name, logo, colours and above all the **time zone**.
- 2 **Staff** — the people visitors ask for. Bulk import from a spreadsheet.
- 3 **Projects** — if contractors sign in against jobs.
- 4 **Visitor types** — which cards the kiosk offers.
- 5 **Settings** → **Visitor form** — which fields each type is asked for.
- 6 **Documents and Induction decks**.
- 7 **Settings** → **Notifications** — so arrivals reach somebody.
- 8 **Devices** — register each tablet and copy its link.
- 9 **Settings** → **Backups** — turn on the off-site copy.

Access levels and accounts

Logins are granted from **Staff**, against a person who already exists there. Five levels:

AREA	RECEPTION	CLERK	MANAGER	ADMIN	OWNER
Dashboard	✓	✓	✓	✓	✓
Visits, roll call, expected, staff list	✓	—	✓	✓	✓
Registry, companies, certificates	✓	—	✓	✓	✓
Projects	✓	—	✓	✓	✓
Reports, printable report	✓	—	✓	✓	✓
Drivers	—	✓	✓	✓	✓
Deliveries	—	✓	✓	✓	✓
Settings, backups, devices, accounts, storage	—	—	—	✓	✓

The rules that override the table

- Anyone signed in can read their own account, change their own password, and read the on-site board's address.
- The **staff list** is readable by anyone who can open a visit — naming who a visit is for needs it. Editing it is administration, because a login can be granted through it.
- Reception can read what paperwork is **lapsing**; changing the certificate rules needs the registry area.
- Anything not classified defaults to administrator, so a route added later is a locked door rather than an open one.

The owner

The first account is the owner: an administrator that cannot be demoted or removed, because an install nobody can reach the settings on is an install nobody can fix. Only the owner can create another administrator, and only the owner can stage a restore.

Temporary passwords

A password set by somebody else is temporary. Until it is changed that login can do exactly two things — read who it is, and change its password — so

whoever typed it does not go on being able to sign in as that person.

When nobody can sign in

There is no reset email to send. The way back in is on the machine holding the data:

```
node scripts/reset-password.js # lists the accounts
node scripts/reset-password.js you@example.com 'a new password'
```

On Railway that is a one-off command against the service with the volume mounted. It re-enables the account and signs out every session it had.

Branding and the site clock

Settings → **Branding** sets the organisation name, the kiosk headline and sign-out message, two brand colours, the time zone and the date format.

The time zone matters more than it looks

It is the site's own, and everything that names a day or a time uses it: the date on a badge and inside the badge number, the "today" counts, the activity and busiest-hour charts, and every arrival time on the dashboard.

Times are stored as UTC, so moving a site to a different zone re-reads the existing history in the new one rather than shifting it. Set this before opening a site and then leave it alone, unless the site itself moves.

Logo and background

Logo

Appears on the kiosk welcome screen, printed badges, the dashboard sidebar and the sign-in page. PNG or SVG with a transparent background works best.

Kiosk background

One photo behind the welcome screen, or several that crossfade on a timer — 8 seconds to 5 minutes, up to 20 photos. A slider controls how much they are darkened so the welcome text stays readable, with a live preview beside it. Landscape, 1600px wide or more. Rotation pauses while somebody is signing in.

Welcome text position

Place the headline, sub-heading and button left, centre or right and top, middle or bottom, so they sit clear of whatever is in the photo.

Uploads are checked by their actual bytes rather than their file extension, so a mislabelled file is rejected instead of becoming a broken image on the kiosk.

Visitor types and the kiosk home screen

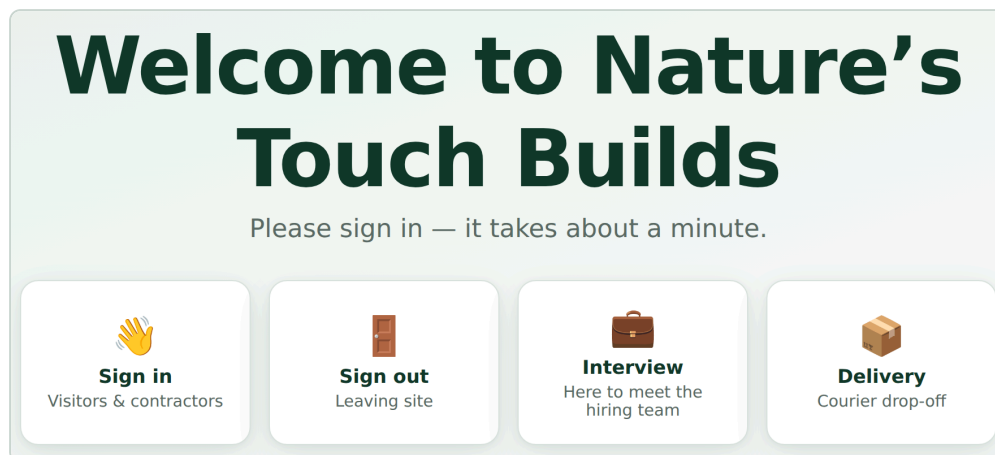
Visitor types is where the kiosk's cards are defined. Each type carries a name in both languages, an icon, and where it appears: its own card on the home screen, an option behind **Sign in**, both, or hidden.

Add a type — a Cleaner card, an Auditor card — and it immediately gains its own form settings, document assignments, induction targeting and per-device placement.

The icon and the order

Press the icon to open a picker of emoji chosen for the job: things that read as a delivery driver or a contractor from six feet away. Anything not in the list can still be pasted in.

The home-screen preview beside the list can be dragged to set the order the kiosk shows the cards in. What you see in the preview is what appears on the tablet.



The home screen this produces. Four cards, each one a visitor type placed on the home screen rather than behind Sign in.

The standard set

Sign in

The general card, offering whichever types sit behind it.

Sign out

Badge scanner first, name search underneath.

Contractor

Straight into a contractor sign-in with no picker: name, company, phone and the project they are on. No vehicle is asked for.

Interview

Candidates, recorded as an interview visit without choosing a type.

Driver

Haulier, vehicle registration, load or order reference, delivering or collecting.

Does not ask who they are visiting.

Delivery

Courier drop-off with a parcel photo.

Request entry

Appears only once a door is set up and kiosk unlock is switched on. If you have ticked it and it is not showing, that is why.

The details form, per type

Settings → **Visitor form** is a grid: fields down the side, visitor types across the top. Each cell is off, optional or required. Hovering highlights the whole row and the whole column, so you can see which field and which type a dropdown belongs to before changing it.

Under **Wording**, rename a field per type — a driver is asked for a haulier, not a company — and add a line of help underneath it.

The order things are asked

Settings → **The order things are asked**. Each visitor type has its own order for the four steps: their details, the photo, documents and questions, and the induction deck. A contractor can watch the induction before signing anything while visitors keep the usual order.

Finding the visitor always comes first, because it decides whether the induction is needed at all. A step that does not apply is skipped wherever it sits.

Changes reaching the tablets

Every saved change bumps a configuration revision. Kiosks check in every 20 seconds and reload when that number moves, so branding, cards, fields, documents and inductions all reach the tablets by themselves. A kiosk part-way through a sign-in waits until the visitor has finished, so nobody has a form rearranged under them mid-typing.

Documents and questions

Each document in **Documents** is assigned to the categories that must sign it, matching the kiosk cards. Deliveries sign nothing.

Typed wording, or an uploaded file

A document is either wording typed into the dashboard or an uploaded PDF or Word file — also ODT, RTF, plain text or an image. Save the document first, then reopen it and use **Upload PDF or Word**. The file is rendered to page images so it is read on the kiosk exactly as drafted; the layout of a safety document is part of what is being agreed to.

An uploaded file replaces the typed wording on the kiosk, and **Remove file** goes back to it with the typed text kept meanwhile. Uploading, replacing or removing a file bumps the document's version, so copies already signed stay exactly as they were signed.

Questions

A document can carry its own questions, asked just above the signature: **Yes / No**, **short answer**, or **choose one** from your own options.

A question can depend on an earlier answer — *Only ask this if* — so a No opens one follow-up and a Yes opens another. A question nobody was shown is never required of them, and if they change the answer above it, the abandoned branch is cleared rather than stored.

A document with questions and the signature switched off is a questionnaire: the visitor answers and carries on. The body text can be left empty.

ANSWERS KEEP THEIR WORDING

Answers are stored against the visit alongside the signature and shown with the wording used at the time, so a later edit to a question never changes what a past answer appears to mean.

Induction slideshows

Create a deck in **Induction decks**, then drop a `.pptx`, `.pdf` or image onto it.

How the file becomes slides, best first:

- 1 **LibreOffice and poppler installed** — each slide rendered to a PNG. Pixel perfect.
- 2 **PowerPoint, no LibreOffice** — the file is unpacked and each slide rebuilt from its text and images. Layout approximate, content all there.
- 3 **PDF, no poppler** — shown as one scrollable embedded document.
- 4 **Images** — one slide each, in upload order.

`/api/health` reports which you have, and the Induction decks page says plainly whether it can render properly.

Who has to watch it

- First-time visitors, always.
- Returning visitors, only if the deck version changed since they last watched, or you set "repeat after N days", or you tick "show it every visit".
- Reset one person from **Visitor registry** → **Open** → **Reset induction**.

Uploading a new file replaces the slides and bumps the deck version, which is deliberate: everyone sees the new induction once, including people who watched the old one.

Two languages

Upload the English and Spanish decks as two decks assigned to the same visitor types, with **Language** set on each. The kiosk plays the one matching the language on screen — switching even at the last moment switches the deck — and falls back to English where no Spanish deck exists. Watching either counts, so nobody sits through the same content twice.

Proving they read it

Set **minimum seconds per slide** and Next shows a countdown that only unlocks when the time is up. A slide already sat through stays unlocked, so re-reading an earlier one costs nothing. Completion is stored per person with the deck version and the seconds taken, and appears on the visit record.

UPLOAD A PDF IF LAYOUT MATTERS

PowerPoint shrinks text that overflows its box and stores only the shrink factor; LibreOffice ignores it and draws the text full size, so a tight slide comes out with words under the pictures. Nothing on the server can undo that. **File** → **Export** → **PDF** bakes the layout in, and the PDF is still split into slides. The same applies to any deck built on a brand font, or on Microsoft's Aptos, for which there is no metric-compatible stand-in.

Badges and printers

Badges are off by default. The **Badges** page holds whether they print at all, the label stock, what the badge shows, a live preview at real size, a test print and reprinting.

Setting up the printer

- 1 Connect the label printer to the device showing the kiosk — USB, or a network/AirPrint printer.
- 2 Make it the default printer on that device.
- 3 Set the label size to match your stock. 62 × 100 mm is a common Brother size.
- 4 In Chrome or Edge print settings: margins **None**, headers and footers **off**, background graphics **on**. On iPad, AirPrint remembers the printer you pick.
- 5 Print a test badge and adjust width, height and font scale until it lines up.

The badge number

Built from tokens, with a live sample beside the field as you type:

TOKEN	BECOMES
<code>{prefix}</code>	The prefix for that visitor type, or the general one
<code>{yyyy}</code> <code>{yy}</code>	The year, four or two digits
<code>{mm}</code> <code>{dd}</code>	Month and day, in the site's time zone
<code>{type}</code> <code>{TYPE}</code>	The visitor type, lower or upper case
<code>{seq}</code>	The sequence for that day, zero-padded

The default `{prefix}{yy}{mm}{dd}-{seq}` gives `NTB260825-004`. Set a different **prefix per visitor type** so contractors and drivers run separate series — each series counts on independently, which is why two people can hold `C...-001` and `T...-001` on the same day without a clash.

GAPS ARE DELIBERATE

The sequence carries on from the highest number already handed out that day, so deleting a visit leaves a gap rather than letting the next arrival be given a number somebody on site is still wearing.

Printers register

Printers records name, model, which roll is loaded and its colour, how the printer is reached, a static IP where there is one, and its location. Each device records which printer sits beside it. How a printer is *reached* matters most, because printing runs over AirPrint:

Network

Printer and tablet on the same Wi-Fi. The normal case.

Wireless Direct

For a tablet on cellular with no site Wi-Fi: the printer hosts its own network and the tablet joins it, keeping internet over LTE. On a Brother QL-820NWB: Menu → WLAN → Wireless Direct → On.

Bluetooth

Inventory only. iPads print over Bluetooth solely from the maker's own app, never from a web kiosk, so a Bluetooth-only printer will not print badges.

Devices

Register every tablet under **Devices**. Each gets:

- Its own kiosk URL — `/kiosk/north-gate-ipad` — with a one-press copy button. Because the device is named in the address, a home-screen icon always comes back as that device.
- Which cards it shows and in what order, independently of every other tablet.
- A location, a default camera, a printer, and whether it prints badges at all.
- An operational mode, and an online status from a heartbeat every minute.

A tablet opened at plain `/kiosk/` works too, but shows every card.

Notifications

Settings → Notifications. Six channels, each independently switchable, each with a test button:

CHANNEL	WHAT IT NEEDS
Email	SMTP host, port, user, password. Any provider. Includes the photo.
Slack	An incoming webhook URL. Photo as a thumbnail.
Microsoft Teams	A Workflows URL. Channel post or a DM to one person.
Google Chat	An incoming webhook URL.
Generic JSON	Any URL. <code>{event, details[], photo_url, timestamp}</code> .
SMS	Twilio SID, auth token and a from-number. Numbers converted to E.164.

Chat notifications go to two places at once: a company channel that sees every arrival, and each person's own webhook for their own visitors. Switch the channel off being always-on and it becomes a fallback for people who have no webhook of their own.

The payload format is detected from the webhook URL, so one host can be on Slack and another on Teams. The format dropdown only applies to URLs that are not recognised.

Which events fire

Arrival, sign-out, induction completed and delivery are separate toggles. SMS has its own arrival and delivery toggles, so texts can be kept for arrivals only.

What has been sent

Settings → Notifications → What has been sent lists the last 50 attempts on every channel with the address, the result, and the reason for any failure — including ones skipped because a channel is off.

A FAILING CHANNEL NEVER BLOCKS A SIGN-IN

The visitor completes, and the reason is recorded against the visit. Failed posts are retried after a minute, five minutes and twenty-five minutes, then given up on — posting an arrival an hour after somebody walked in is worse than not posting it.

TEST MESSAGES CANNOT REACH A REAL PERSON

Tests go to the address in **Send test emails to** and nowhere else. Secrets — the SMTP password, the Twilio token — are write-only: they come back masked, and the mask is never written back over the real value.

Designing the chat card

What lands in Teams is designed rather than fixed, per event and per visitor type.

Per event

Four events carry their own card: **sign-in**, **sign-out**, **induction finished** and **parcel arrived**. For each you choose:

- The title and sub-heading, written as templates with the visitor's name, company, host and time filled in.
- Which fields appear, and in what order.
- Whether the photo shows, its size, its shape, and whether it sits beside the heading or above.
- The header colour, whether details read as facts or lines, and the footer.
- Whether the host is tagged, and the wording of the tag.

A live preview shows the card as Teams will render it, so nothing has to be tested by making somebody sign in.

anybody setting up a link of their own. A staff member who wants their own direct message can still paste a personal Teams link on their record.

☒ **Post every arrival to the company channel**
With this off, the channel is only used for people who have no Teams link of their own

Company channel link

Format for unrecognised URLs
Microsoft Teams ▼

A Teams, Slack or Google Chat link is recognised on sight; this only applies to anything else.

▼ **Getting the Teams link for a channel**

1. In Teams, hover the channel → ... → **Workflows**.
2. Choose the template **"Post to a channel when a webhook request is received"**.
3. Name it "Smart Lobby", confirm the team and channel, then **Add workflow**.
4. Copy the HTTPS URL it shows — you only get it once — and paste it above.
5. Save, then press **Send test to Teams** below.

For an individual person, the same thing with the *chat* template, pasted into their record on the **Staff** tab. Full instructions for both, including what to do when your tenant hides the chat template, are on that tab under **Setting up a chat webhook**.

WHAT THE MESSAGE LOOKS LIKE

Four different things get announced, and they are not the same kind of message — an arrival wants a face and a project, a sign-out wants a time, a parcel has no visitor on it at all. Each gets its own design. The preview is built by the same code that sends the real thing, so what you see here is what lands in the channel.

Sign-ins Sign-outs Finished site induction Parcel arrives

When a visitor signs in at the kiosk

Every type Visitor Contractor Interview Driver

This is the card every visitor type gets. Pick a type above to give that one its own.

Heading colour Blue ▼

Teams only offers its own palette, so this is a choice rather than a colour picker

Details layout Two columns — label beside value ▼

Heading
{name} has arrived to see {host}

Under the heading
{company}

Footer

Preview

The card designer. The controls on one side, the card as Teams will render it on the other — so nothing has to be tested by making somebody sign in.

Per visitor type, on top of that

Any visitor type can carry its own card for any event. A contractor arrival can look nothing like an interview arrival. Types without their own card fall back to the event's design, so you only override what actually differs.

Quick links

Up to four buttons on the card, opening the dashboard, the on-site board, the roll call or that visitor's own record — so somebody reading the message in Teams can act on it without hunting for the tab.

Who else gets tagged

Beyond the person being visited, any visitor type can be routed to other staff — a safety officer who wants every contractor, an HR manager who wants every interview. They are picked from checkboxes against the staff you already have, are tagged in the post, and are sent it directly if they have a webhook of their own. Staff marked inactive are dropped automatically.

TAGGING ONLY WORKS WHEN TEAMS CAN RESOLVE THE PERSON

A mention needs a name and an email address Teams recognises. Somebody with no email on their staff record is not tagged, because a mention Teams cannot resolve renders as raw markup — which is worse than no tag at all.

Doors and access control

Each door is one HTTP call. That covers smart relays directly — Shelly, Tasmota, ESPHome, Home Assistant — and reaches a proper access panel (Honeywell, Paxton, Net2) through one: Smart Lobby calls a relay, the relay closes a contact across the door's REX or auxiliary input, and the panel releases the door as if somebody had pressed the exit button. The panel keeps its own schedules, interlocks, fire release and log, and needs nothing added to its software.

Add a door in **Access & doors** with a URL, method, optional headers and body. These placeholders are filled in at unlock time: `{{seconds}}`, `{{door}}`, `{{actor}}`, `{{visit_id}}`, `{{timestamp}}`.

Shelly relay	GET	<code>http://192.168.1.50/relay/0?turn=on&timer={{seconds}}</code>
Home Assistant	POST	<code>http://192.168.1.10:8123/api/services/lock/unlock</code> Headers: <code>{"Authorization":"Bearer YOUR_TOKEN"}</code> Body: <code>{"entity_id":"lock.front_door"}</code>

Doors fire automatically on sign-in, on sign-out, from a **Request entry** button on the kiosk, or from **Test unlock** in the dashboard. Every attempt is logged with who, when, why and the result.

A door can be set up before it is wired: add it, write the panel, door and terminals into its wiring notes, and leave it disabled. It stays listed, is offered on no kiosk, and is never called until you tick Enabled.

When an unlock fails, the result says which end to go and look at — whether the relay refused the connection, could not be found, sat on a network this server cannot reach, never answered, or answered and turned the request down. A 401 or 403 is usually a token in the headers.

Projects, companies and certificates

Projects

The jobs a contractor can be on site for, each with a name, an optional Spanish name and a short code. Contractors pick one at sign-in; the project lands on the visit record, in the visits CSV, and drives hours-per-project on the reports.

Close a finished job by switching **Active** off: it disappears from the kiosk but keeps its history. A project that has ever been signed in against cannot be deleted, only closed.

Companies

Firms are records rather than free text. Because visitors type their own company name, the same firm arrives spelled several ways — the page flags near-duplicates (one letter apart, or the same name with a suffix like Ltd or LLC) and offers to merge them. Renaming fixes a misspelling everywhere at once, including on the visits that recorded it.

Companies

Every firm that has been on site. A name typed wrong at the kiosk is corrected here, and two that are the same firm can be merged — both fix every visit already recorded, not just the next one.

Add a company

COMPANY	PEOPLE	VISITS	LAST ON SITE	STATUS			
Ashcroft Surveying	1	1	30 Aug, 09:31	allowed	Edit	Merge	Remove
Bellman Freight	1	1	30 Aug, 11:27	allowed	Edit	Merge	Remove
Bevan & Locke	1	1	30 Aug, 09:38	allowed	Edit	Merge	Remove
Halden Plumbing	2	2	30 Aug, 08:10	allowed	Edit	Merge	Remove
Ironbark Steel	3	3	30 Aug, 09:24	allowed	Edit	Merge	Remove
Kestrel Groundworks	2	2	30 Aug, 07:42	allowed	Edit	Merge	Remove
Longhaul Logistics	2	2	30 Aug, 13:34	allowed	Edit	Merge	Remove
Marlow Scaffolding	2	2	30 Aug, 08:56	allowed	Edit	Merge	Remove
Meridian Design	1	1	30 Aug, 10:52	allowed	Edit	Merge	Remove
Northgate Roofing	1	1	30 Aug, 11:59	allowed	Edit	Merge	Remove
Redgate Insurance	1	1	30 Aug, 10:45	allowed	Edit	Merge	Remove
Vega Electrical	3	3	30 Aug, 09:17	allowed	Edit	Merge	Remove

Barring a company turns away everybody from it at the kiosk, with the same “please see reception” a barred individual gets. Removing one leaves its people and their history alone — they simply stop being attached to a firm.

Companies. Real records rather than free text, with names close enough to be worth a look offered for merging.

Certificates

Insurance, CSCS cards, method statements — held against a person or a company, each with an expiry date. Configure which kinds are required for which visitor types under **Settings** → **Certificates**.

A missing or expired certificate can either warn the desk at sign-in or stop the visitor at the kiosk, depending on how you set it. The dashboard shows what is lapsing so it can be chased before somebody is turned away at the gate.

Backups and restoring

A backup nobody has ever opened is a promise, not a copy. This chapter is about turning it into a fact before you need it.

What runs on its own

A ZIP holding the database *and* every uploaded file is written nightly to `data/backups/`, seven kept, each opened and integrity-checked after it is written. The first runs a minute after the server starts, so a fresh deploy is never a day away from having any backup at all.

THEY ARE ON THE SAME VOLUME AS THE DATA

On their own they answer "something corrupted the database", not "the volume is gone". For that you need a copy somewhere else.

The off-site copy

Each new backup can be posted straight into a **OneDrive** folder. The destination is a Power Automate flow — "When an HTTP request is received" → "Create file" — which needs no Azure app registration and no admin consent. Step-by-step instructions are on the Backups page.

Power Automate refuses a request over about 90 MB, which a site keeping ninety days of photos passes inside a fortnight. Past that size the archive is cut into numbered pieces — `...-1of4-database.zip`, then the files — each a valid archive on its own. The database goes first, so the most important piece is away even if a later one fails, and the pieces are numbered so a gap in the folder is obvious.

Testing a backup without restoring it

Every backup in the list carries a **Test** button. It opens the archive, reads the database inside it and reports whether it would actually restore:

- The database passes SQLite's own integrity check and holds accounts, so somebody could sign in afterwards.
- It carries every table this version of the app reads, so a restore does not land on a schema from before a migration.
- The photos and signatures the records point at are actually inside the archive. This is the one that fails quietly: a database-only copy restores perfectly and every face comes back as a broken image.

Nothing is written and nothing live is touched. Run it occasionally — a backup you have tested is the only kind worth having.

Restoring

- 1 Upload the archive on the Backups page. It is checked before anything happens.
- 2 The current data is backed up first, in case the restore was itself the mistake.
- 3 The restore is **staged**, not applied — swapping the file out from under an open SQLite handle is how a restore becomes a second disaster.
- 4 Restart the server. The staged archive is applied on the way up.

A files-only piece of a split backup is different: it holds no database, so there is nothing to swap and it is unpacked immediately.

Disk space and retention

A full volume has no graceful failure. The database stops writing, the kiosk turns people away, and the backup that would have warned you cannot be written either.

Knowing where you stand

The Backups page shows what is using the volume — photos and signatures, the database, backups kept locally — with the percentage used and an estimate of **how many days the space left will last**, worked out from the last thirty days of arrivals at this site rather than a guess at an average one.

One ZIP holding the database **and** every uploaded file — visitor photos, signatures, deck slides, your logo — written every night, the last seven kept. Each one is opened and checked after it is written, because an unverified backup is a guess.

They sit on the same volume as the live data, so on their own they answer “something corrupted the database” and not “the volume is gone”. **Download one and keep it somewhere else** — that is the copy that survives losing the machine.

Back up now

1 kept, 27 kB in total

TAKEN	SIZE	HOLDS	
30 Aug, 15:46	27 kB	Database and files	Test Download Delete

ROOM ON THE DISK

When this fills, sign-ins stop and no backup can be written — including the one that would have told you.

88% of 258019.6 MB used — 30306.6 MB left.

Photos and signatures — 25 files	13 kB
Database	3.4 MB
Backups kept here — 1 files	27 kB

Photos alone are 11 kB across 22 files. How long they are kept is set under **Data retention & privacy**; shortening it is usually the quickest room to find.

When it gets tight

Rather than let the disk fill and take the kiosk down with it, the oldest photos can be dropped early. They are the least useful thing on the disk and by far the largest — and losing one costs a look-up nobody was going to do, against a site that stops taking sign-ins.

☒ Drop the oldest photos before the disk fills

Start when the disk is this full (%)

Clear back down to (%)

Never touch photos newer than (days)

90

75

14

Free up room now

Has not needed to run — it starts at 90% full.

COPY EACH BACKUP TO ONEDRIVE

A backup sitting on the same volume as the data does not survive losing the volume. This posts each new one straight into a OneDrive folder as it is written, so there is always a copy somewhere else without anybody remembering to do anything.

Room on the disk. What is using the volume, how long the space left will last at this site's rate, and the thresholds the valve works to.

The pressure valve

Under **When it gets tight**, past a threshold the oldest photos are dropped down to a target. Three things it will not do: touch anything inside the floor window (the last fortnight by default), run at all when switched off, or delete quietly — every sweep goes to the audit log with what went and how much it freed.

The visit itself always stays; only the photo on it goes. **Free up room now** runs it on demand, still respecting the floor.

SETTING	DEFAULT	WHAT IT MEANS
Start at	90%	How full before anything is dropped
Clear down to	75%	Far enough below to buy weeks, not another sweep tomorrow
Never touch newer than	14 days	The window in which somebody actually looks a face up

Retention

Settings → **Data retention** runs daily and has three separate windows:

Photos

Deleted after N days. The visit stays.

Licence details

A shorter window of their own — a licence number is the most identifying thing held here, and useful for far less time than the visit record it sits on.

Visit records

Deleted after N days, along with anything in the deleted-records archive past the same window, so the archive never becomes the one place old visitor data lives on.

Visitor photos and signatures sit behind the admin login; only the logo and induction slides are public. The kiosk shows a privacy notice on the details screen, where the asking happens, rather than burying it in a document nobody reads.

Deleting does not destroy

Deleting a visit or a visitor writes the whole record — children and all — to a deleted-records archive first, so one mis-click does not destroy the proof a contractor read the site rules. It can be looked at and put back.

Security posture

- **Sessions** are cookies — HttpOnly, SameSite=Lax, and Secure whenever the request arrived over HTTPS, read from the request itself rather than from an environment variable a host may not set.
- **Passwords** are bcrypt hashes. A password set by somebody else is temporary until changed.
- **Every request** is checked against the role map on the way in. Hiding a menu item from somebody who can still call the endpoint behind it is not a permission system.
- **Rate limits** sit on everything open to the internet: kiosk lookups, searches, sign-in writes, the board, and photo links.
- **Visitor photos** are never public. The kiosk's sign-out list gets a short-lived signed link that works only for that visit, only while they are on site.
- **Uploads** are checked by their actual bytes, not their extension.
- **Secrets** come back from the API masked, and the mask is never written back over the real value.

Hardening the deployment

- Serve over HTTPS. On Railway that is automatic.
- Keep `PUBLIC_URL` accurate — photo links in notifications are built from it.
- Give each person their own login rather than sharing one, so the audit log names a person.
- Reissue the on-site board key if its address gets out further than intended.
- Keep an off-site backup, and test one occasionally.

Tests

```
npm test                # every suite
npm test -- board card  # only suites whose name contains one of these
npm test -- --keep      # leave the test database behind to poke at
```

Each suite gets a freshly started server against a throwaway database, because the rate limiters are per-process and suites sharing one would start tripping them and failing for reasons that have nothing to do with the code. The order in `test/run.js` is deliberate rather than alphabetical.

The browser suites need Playwright and are skipped, loudly, without it:

```
npm install -D playwright && npx playwright install chromium
```

Among the suites are an **attack** suite that tries the things an attacker would and fails if any of them work, and a **day** suite that runs forty people through one kiosk in a single day and checks that the dashboard, roll call, on-site board, reports, paged list, CSV export and printed page all agree about them.

Troubleshooting

The dashboard says storage is not persistent

No volume is mounted. Everything will be erased on the next deploy. Mount one at `/data` before doing anything else.

Teams returns a 400

Workflows requires an Adaptive Card; the older MessageCard format is refused. Smart Lobby sends Adaptive Cards, so a 400 usually means the URL is a legacy Office 365 connector rather than a Workflows one. Create the flow again from "When an HTTP request is received".

An @-mention shows as raw markup in Teams

The person has no email address on their staff record, or the address is not one Teams can resolve. Add it under **Staff**.

Slides come out with words over the pictures

A font the deck used is missing on the server, so the text reflowed. Export the deck to PDF from PowerPoint and upload that instead — a PDF carries its fonts inside it and is still split into slides.

The camera never asks for permission on a tablet

Open `/check/` on that tablet. If it is a third-party kiosk browser, it is almost certainly not forwarding the request. Use Safari with Guided Access.

Badge prints blank or half off the label

Margins must be None, headers and footers off, background graphics on. Then adjust the label size and font scale on the Badges page and print a test.

"Request entry" is ticked but not showing on the kiosk

The card appears only once a door exists and is **enabled**. A door left disabled for wiring is never offered.

Two visitors with the same badge number

Should not happen — the sequence carries on from the highest number issued that day. If it does, check whether the badge prefix was changed mid-day, which starts a fresh series.

A restore did nothing

It is staged, not applied. Restart the server; it is applied on the way up.

The disk is filling

Look at the breakdown on the Backups page. Photos are almost always the answer. Shorten the photo retention window, or leave the pressure valve to handle it — and check that local backups are not the second-largest thing on the volume.

Day-to-day operation of the desk — expected visitors, deliveries, drivers, the roll call — is in the Front Desk Guide.